chapter : 02
Architectural Details

Assembly Language Programming

An assembly language is a type of programming language that translates high-level languages into machine language. It is necessary bridge between software programs and their underlying hardware platforms. Assembly language relies on language syntax, labels, operations, and directives to convert code into usable machine instructions.

8085 microprocessor Architecture:

- 8-bit general purpose microprocessor
- capable of addressing 64K of memory (with 16 address lines).
- Has 40 pins DIP (Dual Inline Package)
- Requires +5V power supply
- Can operate with maxm 3MHz single phase clock and minimum 500 KHz clock.
- 8085 upward compatible (its instructions and features are available in 8086)
- It provides 74 instructions with 5 addressing modes.

Pin-diagram of 8085 up.

1 → X_1
2 → X_2
3 → RESET OUT
4 → SOD
5 → SID
6 → TRAP
7 → RST 7.5
8 → RST 6.5
9 → RST 5.5
10 → INTR
11 → INTA
12 → AD_0
13 → AD_1
14 → AD_2
15 → AD_3
16 → AD_4
17 → AD_5
18 → AD_6
19 → AD_7
20 → V_{SS} → ground reference.

(12 → 19: multiplexed Address Data Bus [P_{12} - P_{19}])

40 → V_{CC} (Power supply +5V)
39 → HOLD
38 → HLDA
37 → CLK (OUT)
36 → RESET IN
35 → READY
34 → IO/$\overline{M}$
33 → S_1
32 → $\overline{RD}$
31 → $\overline{WR}$
30 → ALE
29 → S_0

(34 → 29: control and status signal [P_{29} - P_{34}])

28 → A15
27 → A14
26 → A13
25 → A12
24 → A11
23 → A10
22 → A9
21 → A8

(28 → 21: Address Bus (P_{21} - P_{28}))

8085 Pin configuration:

↳ Pins of 8085 are grouped in 6 categories.

1. ADDRESS BUS (P21 - P28):

→ 16 signal pins
→ These lines are split into 2 segments A15 - A8 and AD7 - AD0.
→ 8 signal lines, A15 - A8 are unidirectional and are used for most significant bits, called higher order address of 16-bit address.

2. Multiplexed Address/Data Bus (P12-P19)

→ They are time multiplexed address and data bus.
→ The signal lines AD7 - AD0 are bidirectional.
→ serve dual purpose:
During execution of instruction cycle, these lines are used as lower address bus.

3. Control and status signals (P29 - P34):

→ This group includes:
2 control signals : $\overline{RD}$ and $\overline{WR}$.
3 status signals : IO/$\overline{M}$, S1 and S0 (to identify the nature of operation) and
1 special signal : ALE (to indicate the beginning of operation

→ If IO/$\overline{M}$ = 1, input-output operation is done.
→ If IO/$\overline{M}$ = 0, memory operation is done.

These signals are combined with $\overline{RD}$ and $\overline{WR}$ control signals generate I/O and memory control signals ($\overline{IOR}$, $\overline{IOW}$, $\overline{MEMR}$, $\overline{M}$

S1	S0	operation
0	0	HLT (Halt, termination)
0	1	write (memory or I/O)
1	0	Read (memory or I/O)
1	1	(opcode fetch)

4. Power supply and frequency

→ V_{CC} : +5V power supply
→ V_{SS} : Ground Reference
→ X_1, X_2 : A crystal RL or RC network.

A frequency is internally divided by 2, so to operate a system at 3 MHz ; the crystal should have a frequency of 6 MHz.

CLK (OUT) [P-37]

→ used as a system clock for other devices.

5. Externally Initiated signals, Including Interrupts:

→ 8085 has 5 interrupt signals, 2 acknowledgement signals and 3 other signals.

Interrupt signals are:

RST 7.5
RST 6.5
RST 5.5
(among them the priority order is 7.5, 6.5 and 5.5) (Inputs)
TRAP (Highest priority) (Input)
INTR (lowest priority) (Input)

$\overline{INTA}$ (output) [P-17]

→ Interrupt Acknowledgement signal.
→ up sends this signal after receiving INTR signal.

READY (Input) [P-35]

→ used to delay up Read or write cycles until a slow responding peripheral is ready to send or receive data.
→ when this signal goes low up waits and as soon as it goes high it reads the data in bus or writes in it.

HOLD (Input)

→ Indicates that a peripheral devices eg: DMA (Direct memory Access) controller is requesting the use of address and data buses.
→ when this signal goes high, microprocessor stops the Buses as soon as the current cycle is completed.
→ up regains it when HOLD signal goes low.

HLDA (output)

→ This is Hold acknowledgement signal and acknowledges the HAD request.

→ Goes low when HOLD signal is removed and up takes over the buses.

RESET - IN [P-36]

→ when the signal on this pin goes low, the program counter (PC) is set to zero.

→ so up is reset.

RESET - OUT [P3]

→ It indicates that the microprocessor is being reset.

→ This signal can be used to reset other devices.

6. Serial IIO devices [P4 and P5]

→ SID (Serial Input Data) to implement
SOD (Serial Output Data) are used ~~for~~ serial transmission

→ In serial transmission, data are sent over a single line, one bit at a time, eg. telephone line.

Internal Architecture of 8085 up:

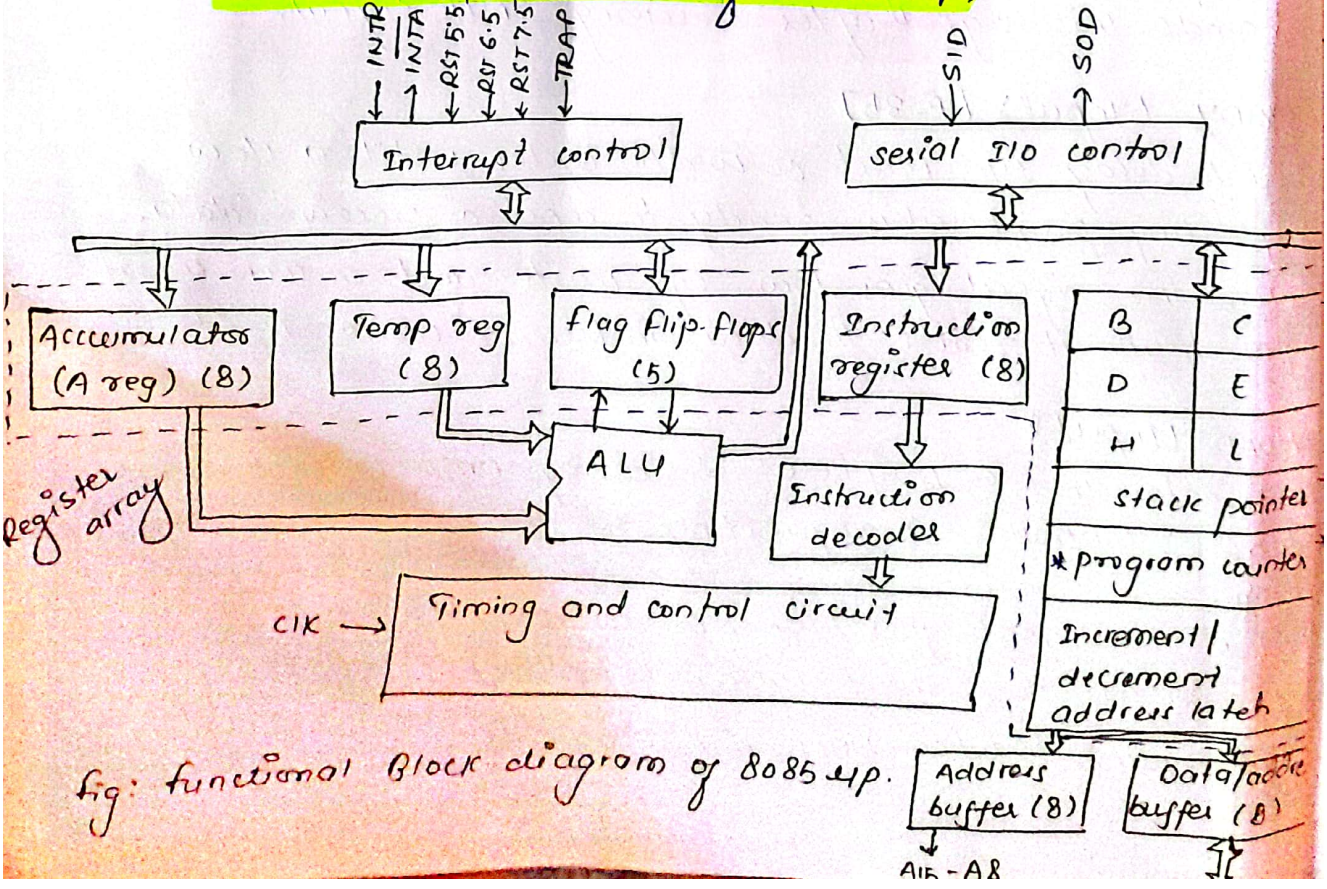


fig: functional Block diagram of 8085 up.

The internal architecture of 8085 µp consists of:

1) ALU (Arithmetic and logical unit)
2) Register Array
3) Timing and control unit (CCU)
4) Instruction Register (IR) and Decoder.
5) Interrupt controller
6) serial I/O control

1. Arithmetic and Logic Unit (ALU):

performs the computing functions. It includes

i) the Accumulator (A) - 8 bit special purpose register used in arithmetic, logic, load and store operations as well as as i/o operations.

ii) The temporary registers (8 bit register used to hold data temporarily during program execution and is not accessible to the programmer.)

iii) The arithmetic and logical circuits

iv) five flag flip-flop registers.

Flag Register:

- It is an 8 bit special purpose register (with 5 bits carrying significant information), that generally reflects the intermediate or final status of the result or conditions of data in the accumulator.
- The result of every arithmetic and logical operations are stored in accumulator and the flags (flip flops) are set or reset according to the result of the operation.
- Thus helps in taking further decisions.

The bit position reserved for different flags in flag register array are as follows:

D7	D6	D5	D4	D3	D2	D1	D0
S	Z	X	AC	X	P	X	CY

fig: Flag Register Array (where, X = don't care conditions)

a) S- sign flag : (for -ve no, S=1 and for +ve no S=0)

→ used for indicating the sign of the data in the accumulator

→ for signed numbers, if D_7 bit is 1, the number is viewed as negative number and if it is 0, the number is considered positive.

→ Thus after the execution of arithmetic or logic operations, sign flag is set for negative value and reset for positive value.

→ In arithmetic operations with signed numbers, bit D_7 is reserved for indicating the sign, and remaining 7 bits are used to indicate the magnitude of a number.

b) Z- zero flag (if result =0, Z = 1, if result ≠0, Z=0)

→ This flag is set if the result of the ALU operations is 0 and reset if the result is non-zero.

→ This flag is modified by the results in the accumulator as well as in other registers.

for eg:

```
  10110011
+ 01001101
-----------
1 00000000
```

Here, Z = 1 as the result of the operation (addition) is 00H.

→ zero flag is also set if data in register becomes 00H after increment or decrement operation.

c) AC- Auxiliary carry flag (AC=1 if carry occurs from D_3 bit to D_4 during addition)

→ This flag is set whenever a carry occurs from lower nibble to upper nibble ie, when a carry is generated from by digit D_3 and passed onto digit D_4.

→ used for BCD operation.

3. P. Parity flag (P=1 for even parity and P=0 for odd)

Parity is defined as the number of 1s in the data in accumulator. After arithmetic and logical operations, if the result has even number of 1s, this flag is set and reset if it had odd number of 1s.

So, parity flag is set for even parity and reset for odd parity.

e.g. for data byte, 0000 0011, P=1, for even no. of 1s although magnitude of no. is odd.

4. C-Carry flag (C=1, for overflow during addition and Borrow during subtraction)

- In an arithmetic operations such as addition and subtraction involving two 8 bit numbers, there is a situation when an add result is overflowing from highest order bit or a borrow may be required in subtraction.
- In both cases, the carry flag is set, otherwise it is reset

```
  1011 0101          1011 0101
+ 0110 1100        - 1100 1100
-----------        -----------
  0010 0001        1110 1001
Carry 1            Borrow 1
```

2. Register Array

- 8085 has 8 and 16 bit registers.
- 8 bit registers are B, C, D, E, H, L, Accumulator and flag
- There are two 16 bit registers
  - is Program Counter (PC)
  - is Stack Pointer (SP) and.
- 8 bit general purpose registers can be used as 8-bit or 16-bit register pair.
- Register pair HL may hold data pointer and is used in register indirect addressing mode.

eg: MOV M, A (copies content of A to the memory contained by HL pair).

General purpose Registers (GPRs) are Accessible to the programmer.

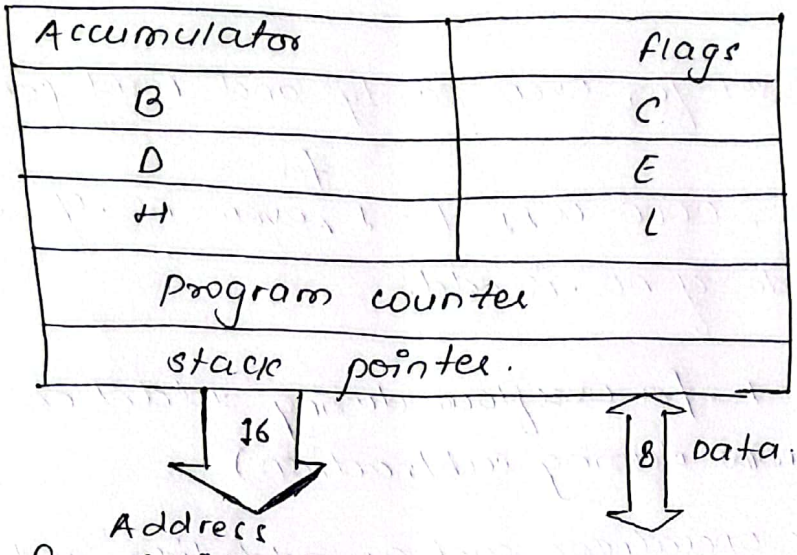


fig: Register array in 8085 up.

Temporary Register W and Z:

- They hold 8-bit data during the execution of some instru
- They are used internally and are not available to the progra er.

Temporary Register (near ALU):

- It is an 8 bit register and is used to hold data for brief time period, during and arithmetic/logic operati
- Not available to programmer.

16-bit Registers:

is Program Counter:

- Deals with sequencing and execution of instruction.
- Contains 16 bit address of the next instruction to be executed.
- Acts as a pointer to the next instruction to be executed
- It is operated by the processor and points instruction afte the processor has fetched the previous one.

i) stack pointer:

Stack is the area of Read/write (RAM) memory used to hold data that will be retrived soon.

The beginning of stack is defined by loading a 16 bit address in the stack pointer (SP).

The stack is usually accessed in FILO/LIFO basis.

stack writing instructions fill the memory positions in progressively decreasing order.

Instructions for stack access: PUSH, POP

During PUSH operation, higher order register content is pushed first.

Eg: PUSH B, then B pushed and then C.

3. Timing and Control unit:

This unit synchronizes all the up operations with the clock and generates the control signals necessary for communication between the microprocessor and peripherals.

Eg: $\overline{RD}$ and $\overline{WR}$ are control signals indicating the availability of data on the data bus.

4. Instruction Register and Decoder:

Coher instruction is feteched from memory, it loaded in the Instruction Register (IR). Then the decoder decodes the instruction and establishes the sequence of events to follow

The instruction register is not programmable and cannot be accessed through any instruction.

5. Interrupt control:

8085 has 5 interrupt signals: TRAP, RST 7.5, RST 6.5, RST 5.5 and INTR.

$\overline{INTA}$ is Interrupt Acknowledgement signal.

valid interrupt may occur for at least 160 ns.

6. Serial I/O control:

↳ The two 8085 pins: SID (Serial Input Data) and SOD (Serial output data) are specially designed for software controlled serial I/O.

↳ Data transfer is controlled through two instructions: SIM and RIM.

↳ These instructions are used for interrupt and serial I/O.

↳ SIM: Set Interrupt Mask [used for serial output data]

↳ RIM: Read Interrupt Mask. [used for serial input data]

↳ Bit D7 of RIM and SIM is seperated for SID and SOD line respectively.

#. 8085 Bus configuration:

The 8085 microprocessor performs its functions, using three sets of communication lines, called buses.

i. The address Bus
ii. The data Bus
iii. The control Bus.

i. Address Bus:

→ It is a group of 16 lines generally identified as A0 to A15
→ The address bus is unidirectional, ie. bits flows in one direction from up to peripheral devices.
→ Thus, MPU uses address bus to identify a peripheral or memory location.
→ In a computer system, each peripheral or memory location is identified by a binary number, called an address, and the address bus is used to carry a 16-bit address.
→ 16 address lines can address upto $2^{16} = 65,536$ memory locations or its memory addressing capability is $2^{16} = 64K$.

(ii) Data Bus:

It is a group of 8 lines used for data flow.

These lines are bidirectional ie. data flow in both directions between the MPU and memory and peripheral devices.

MPU uses data bus to transfer information.

The eight data lines enable the MPU to manipulate 8-bit data ranging from 00 to FF. ($2^8 = 256$)

The largest number that can appear on data bus is $(1111\ 1111)_2 = (FF)_H = (255)_{10}$

(iii) Control Bus

It is comprised of various signal lines that carry synchronization signals.

MPU uses such lines to provide timing signals.

MPU generates specific control signals for every operation (such as memory or I/O Read or write) it performs.

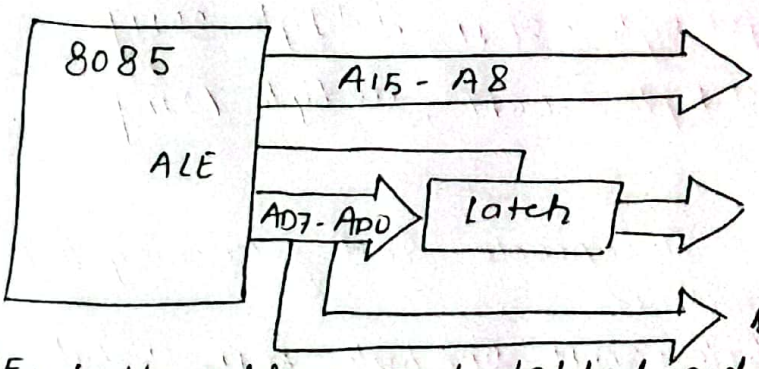
Demultiplexing AD7 - AD0 in 8085 microprocessor:

AD7 - AD0 lines in 8085 µp serve a dual purpose ie. A7 - A0 as address bits and D7 - D0 as data bits.

So they need to be demultiplexed to get all the information.

The higher order bits of the address remain on the bus for 3 clock periods, but lower order bits remain only for 1 clock period and they would be lost if they are not saved externally.

So to make them available for entire 3 clock period, the external latch is used and ALE signal is used to enable this latch.



When ALE = 1, the address can be latched and when ALE = 0, the address is saved.

8085 Instructions

- An instruction is a command to the µp to perform given task, on specified data.
- set of instruction is known as program.
- Each instruction has 2 part: i) opcode
  ii) operand
- opcode (operational code) represent the task to be perf and operand is the data to be operated on.

eg: ADD B [redacted].
↓ opcode.

MOV C, A
↓ opcode: [redacted]

Instruction word size:

- 8085 instruction set is classified into the following groups according to word size or byte size:
  i) 1-byte instructions.
  ii) 2-byte instructions
  iii) 3-byte instructions

(1) 1-byte instructions:

→ It includes the opcode and operand in the same byte.

eg: MOV C, A [byte = 4FH]
ADD B [byte = 80H]
CMA [byte = 2FH]

(2) 2-byte instructions:

→ Here 1st byte specifies the operation code and 2nd byte specifies the operand.

eg: MVI A, 32H [1st byte = 3EH, 2nd byte 32H]
MVI B, F2H [1st byte = 06H, 2nd byte F2H]

(3) 3-byte instruction:

→ Here 1st byte specifies the opcode and following 2-bytes specifies the 16-bit address.

eg: is LDA 2050H [1st byte: 3AH, 2nd byte = 50H 3rd byte = 20H]

8085 Instruction sets

Since the 8085 is an 8-bit device it can have upto (256) instructions. However, the 8085 only uses 246 combinations that represent a total of 74 instructions. Most of the instructions have more than one format.

8085 instructions can be grouped into 5 different groups.

1) Data transfer (copy) Instructions
2) Arithmetic Instructions
3) Logical and Bit manipulation Instruction
4) Branch Instructions
5) Machine control Instructions.

1) Data transfer Instructions:

These instructions performs the following 6 operations:

- Load an 8-bit number in a register
- Load 16 bit number in a register pair.
- copy from register to register
- copy between register and memory
- copy between I/O and accumulator
- copy between register and stack memory.

They are:

1) MVI Rd, data [load 8-bit data in register]
   eg: MVI B, 4FH

2) MOV Rd, Rs [copy data from source register Rs into destination register Rd]
   eg: MOV B, A

3) LXI Rp, 16-bit [loads 16-bit number in a defined register pair]
   eg: LXI B, 2050H

4) OUT 8-bit (port address) [send (write) data byte from the accumulator to an output port / device]
   eg: OUT 05H.

v) IN 8-bit (port address) [Accepts (reads) data byte from input port / device and place it in the accumulator]

eg: IN 07H.

vi) LDA 16 bit memory address [copy the data byte into A the memory specified by 16-bit memory address]

eg: LDA 2050H.

vii) STA 16 bit memory address [copy the data byte into me specified by 16-bit address from A]

eg: STA 2070H.

viii) LDAX Rp [copy the data byte into A from the memo specified by the address in the register pa

eg: LDAX B

ix) STAX Rp [copy the data byte from A into the memory specified by the address in the register pair

eg: STAX D

x) MOV M, R [copies the content of specified register to th memory location specified by HL pair]

eg: MOV M, B

xi) MOV R, M [copies the content of memory location specifi by HL pair to the register specified]

xii) SHLD address [store H and L Register Direct]

eg: SHLD 2050H (here, assume HL register contains 01H a FFH respectively.

then after executing above instruction, 2050H = FFH and 2051H = 01H respectively.)

xiii) LHLD address [load H and L Register Direct]

eg: LHLD 2050H (Here assume 2050H = FFH and 2051H = 01H respectively then after execution of above instruction [H] = 01H and [L] = FFH respectively.]

2. Arithmetic Instruction

The main arithmetic operations performed by arithmetic instructions are:

i) Addition
ii) Subtraction
iii) Increment / Decrement [by 1]

Addition:

1) ADD R [add the contents of specified register to the content of accumulator (A) and store the final result in A.]

eg: ADD B [let A = 93H, now result after ADD B, in A = 14AH so B = B7H Flag set, S=0, Z=0, and CY=1]

2) ADI 8 bit data [Add 8-bit data to the contents of A]

eg: ADI 90H

3) ADD M [Add contents of memory to A, the address of memory is in HL register]

eg: ADD M.

4) ADC R [Adds the content of specified register to A including carry]

eg: ADC B

5) ADC M [Adds the content of memory address stored in HL pair to the content of A with carry]

eg: ADC M

6) ACI 8-bit data [Adds with carry the 8-bit data in the instruction to A]

eg: ACI 33H

7) DAD Rp [Adds contents of specified register pair to the content of HL pair ie. used in 16 bit addition]

eg: DAD B (suppose initially [HL] = 0242H and [BC] = 0123H) after this instruction result in [HL] = 0242H + 0123H = 0365H

Subtraction

i) SUB R [subtracts the content of specified register from the content of accumulator (A) and store the result in A]

eg: SUB B

ii) SUI 8-bit data (subtract immediate data from accumulator)

iii) SBB R (subtract memory from accumulator with Borrow)

iv) SUB M (HL pair की memory address की data को accumulator की data में से subtract करके)

v) SBI 8-bit Data (Immediate Data को Accumulator की data में से subtract करो with Borrow)

Increment / Decrement:

i) INR R ; increment content of R by 1.

ii) INR M ; increment content of HL pair by 1.

iii) INX Rp ; [e.g. INX B : increment the content of BC by 1]

iv) DCR R ; Decrement content of R by 1.

v) DCR M ; Decrement content of HL pair by 1.

vi) DCX Rp ; [e.g: DCX B; decrement the content of BC by 1]

3. Logical and Bit manipulation Instructions:

These instructions perform:

i) AND

ii) OR

iii) X-OR (exclusive OR)

iv) Compare

v) Rotate Bit

→ These instructions implicitly assume that the accumulator is one of the operand and place the final result in A.

→ The various instructions available under this category are

ANA R ; logically ANDs the content of specified register with the content of A and store the final result in A

ii) ANI 8-bit data ; logically ANDing content of A and the immediate data and store the final result in A.

ii) ANA M ; logically ANDing the content of HL pair data with A.

) ORA R ; logically ORs the content of specified register to A

) XRA R ; Exclusive ORs the content of R and A

) XRI 8-bit data

ubi) XRA M

logical instructions are used for masking (hide certain) bit operation.

gs: ANI 80H (masks all other bits except MSB)
ANI 01H (masks all other bit except LSB)

Compare Instructions: [comparision = A - R]

) CMP R
) CPI 8-bit data
) CMP M

for these instructions:
- If A > R, CY and Z flags are reset
- If A < R, Cy is set and Z is reset
- If A = R, Z is set and Cy is reset

so here we compare the data by checking the carry flags after executing these instructions.)

Complement (logical NOT or does 1s complement):

) CMC (complements Cy flag)
) CMA (complements contents of Register A)

Other Bit-manipulation instructions:

RAL ; Rotate Accumulator left through carry - bit D7 is placed in the carry flag and carry flag is placed in LSB (9-bit rotation).
otate acc. with carry)

for RAR ; Rotate Accumulator Right through carry - bit D0 is placed in the carry flag and carry flag is placed in MSB (9-bit rotation)
otate acc. with carry)

) RLC ; Rotate Accumulator left - D7 bit is placed in D0 as well as carry flag (8-bit rotation).
e:

) RRC ; Rotate accumulator Right - D0 bit is placed in D7 as well as in carry flag 8-bit rotation.

CY — — — — — — — — —
MSB LSB.

Note: RLC can be used to multiply a number by 2 and RRC can be used to divide a number by 2.

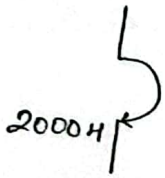
4. Branch Instructions:

→ These are used to change the program sequence.

Unconditional:

i) JMP 16-bit address ; unconditional jump to specified address.

eg: JMP 2000H



conditional:

ii) JZ 16-bit address ; Jump to specified address if zero flag is set (if Z=1) or result is

iii) JNZ 16-bit address ; Jump to specified address if NOT zero (if Z=0).

iv) JC 16-bit address ; Jump to specified address if carry (ie. if carry flag . CY=1)

v) JNC 16-bit address ; Jump to specified address if NOT carry (ie. carry flag . CY=0)

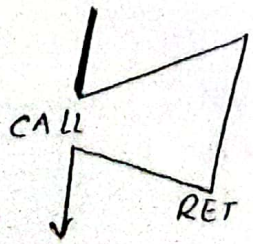
vi) JP 16-bit address ; Jump to specified address on positive (ie. if sign flag . S=0).

vii) JM 16-bit address ; Jump to specified address on Negative (minus) (ie. if sign flag is S=1)

viii) JPE 16-bit address ; Jump on even parity (ie. if parity flag

ix) JPO 16-bit address ; Jump on odd parity (ie if parity flag

x) CALL 16-bit address ; subroutine call



Note:
Higher byte → High

iii) RET : Return to main program from the subroutine

v) Machine control Instructions:

i) HLT ; stop and wait for the next program
ii) NOP ; No operation.

#. Addressing Modes in 8085 MP:

The various ways in which a processor can access data, during program execution are referred to as its addressing modes. In other word, the way in which the operand is specified is called its addressing modes. Best selection of addressing mode helps in reducing the computational complexity in any program.

In 8085 microprocessor, different 5 types of addressing modes are available. They are:

a) Immediate Addressing Mode:

The data is present in the instruction itself. operands can be of 8 or 16-bits in size.

Eg: MVI A, 33H ; Loads the 8 bit data 33H in accumulator registers.

LXI B, 2050H ; loads the 16-bit data 2050H in BC register pairs

ADI, 20H ; add content of A and 20H

ANI, 80H ; mask all bit except MSB.

b) Direct Addressing Mode:

In this addressing mode, the effective address of memory or I/O port/device is defined in the instruction.

Eg: LDA 2050H ; Loads the data stored in memory location 2050H to accumulator.

STA 2070H ; stores the data from accumulator to the memory location 2070H.

IN 20H ; Reads (loads) the content of input port 20H to accumulator

OUT 30H ; sends (stores) the content of accumulator to the output port 30H

SHLD 2050H ;

3. Register Addressing Mode:

In this addressing mode, registers are used to define data, ie both source and destination.

eg: MOV A,B ; copies the content of Register B to Accumulator

ADD D ; Adds the content of Accumulator and D register and store the result in Accumulator

ANA B ; Logical AND between A and B

DAD ; Adds content of specified register pair to HL. content of HL pair is used for 16 bit addition

4. Register Indirect Addressing Mode:

In this addressing mode, the register pair are used to provide the memory address of data.

eg: LDAX B ; copy the data byte into A from the memory specified by the address in the register pair BC

STAX D ; copy the data byte from A to the memory specified by the address in the register pair DE.

MOV M, A ; copy the content of A into HL pair/memory specified.

5. Implied Addressing Mode: (Hidden)

→ In this addressing mode, the operand for the instruction is hidden and it is normally the Accumulator (A).

eg: CMA ; complement (1s complement) the value of accumulator.

RRC ; Rotate accumulator Right - D0 bit is placed in as well as in carry flag.

RLC ; Rotate accumulator Left - D7 bit is placed in D0 as well as in carry flag.

XCHG; Exchange the content of DE and HL pair.

HLT ; Terminate the program.

Register transfer language [RTL]

The symbolic notation used to describe the microoperation transfers among registers is called as Register transfer language (RTL). The term 'Register transfer' implies the availability of hardware logic circuits that can perform a stated micro-operation and transfer the result of operation to the same or another register. And the 'Language' refers to programming language that denotes the procedure for writing symbols to specify a given computational process. Thus, a register transfer language is a system for expressing in symbolic form the microoperation sequences among the registers of a digital module.

In RTL, the registers are generally denoted by capital letters and the information transfer from one register to another is designated in symbolic form by means of a replacement operator.

The general example of RTL can be as:

$R_1 \leftarrow R_2$

This statement denotes a transfer (replacement) of content of register R_2 into R_1 but the content of R_2 doesn't change after transfer.

```
MOV A,B     ; A ← B
ADD B       ; A ← [A] + [B]
```

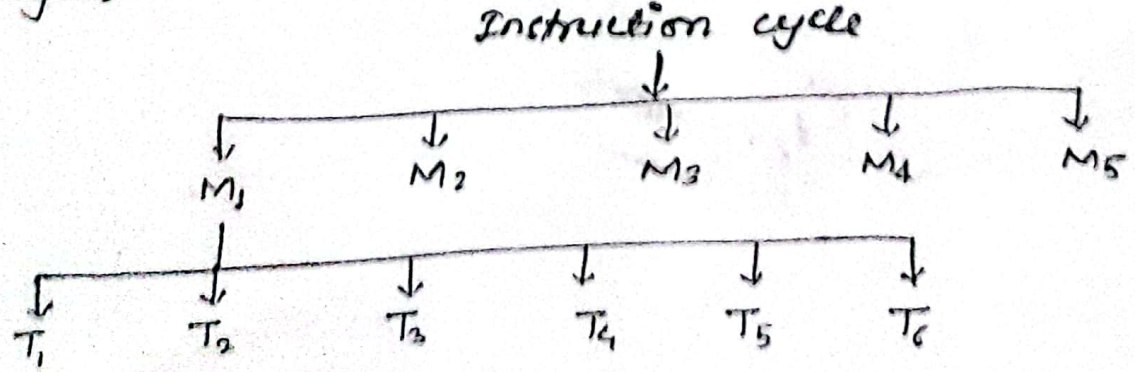
TIMING DIAGRAM 8085

- Timing diagram is a graphical representation of instruction execution in steps with respect to the clock.
- The execution time is represented in T-states
- The following control signals, status signals and buses must be shown in the Timing diagram:
  - → Higher order address Bus
  - → lower order address Bus
  - → ALE
  - → $\overline{WR}$
  - → $\overline{RD}$
  - → IO/$\overline{M}$
  - → S_1 and S_0

T-State: Defined as one sub-division of operation performed in 1 clock period. 1 T-state is equal to one clock period.

Machine cycle:
- Time required to perform 1 operation may be opcode fetch, memory read, memory write or I/O read or I/O write is called machine cycle.
- consists of 1 to 6 T states.

Instruction cycle:
- Time req'd to execute an instruction is called instruction cycle
- In 8085 µP, the instruction cycle consists of 1 to 5 machine cycles.



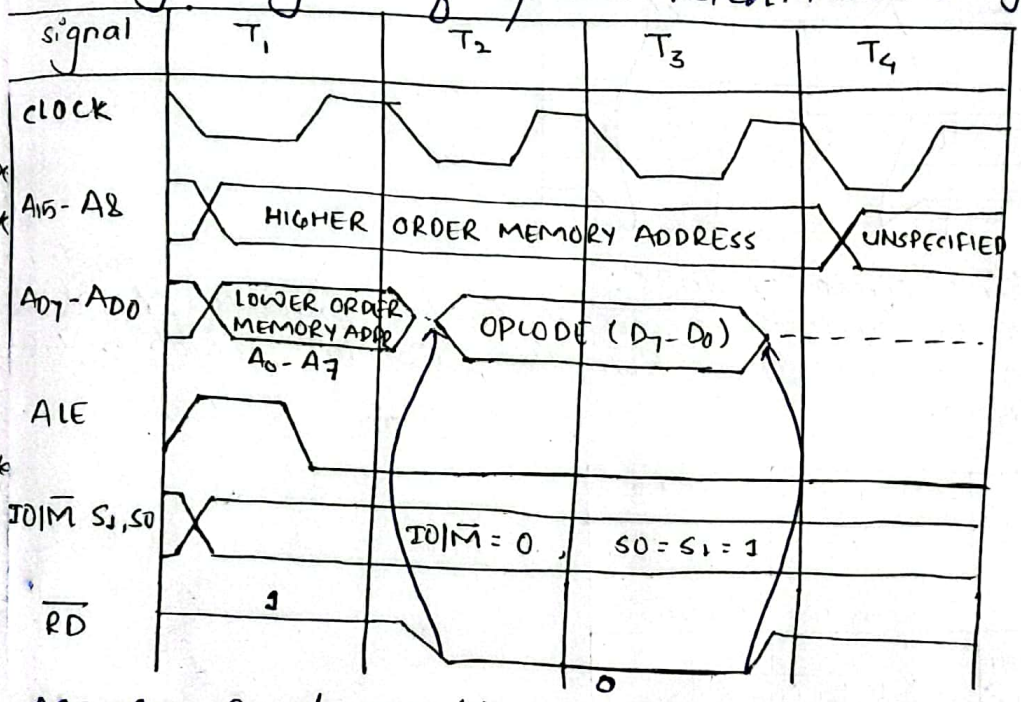
where, M_n – Machine cycle
T_n – T state.

combination of status signals.

Machine cycle	IO/$\overline{M}$	S_1	S_0
opcode fetch	0	1	1
Memory read	0	1	0
Memory write	0	0	1
I/O read	1	1	0
I/O write	1	0	1

Note:
→ 1 byte instruction के लिए only opcode fetch
→ 2 byte instruction : OF + MR
→ 3 byte instruction : OF + MR + MR

1. Timing diagram of opcode fetch machine cycle:

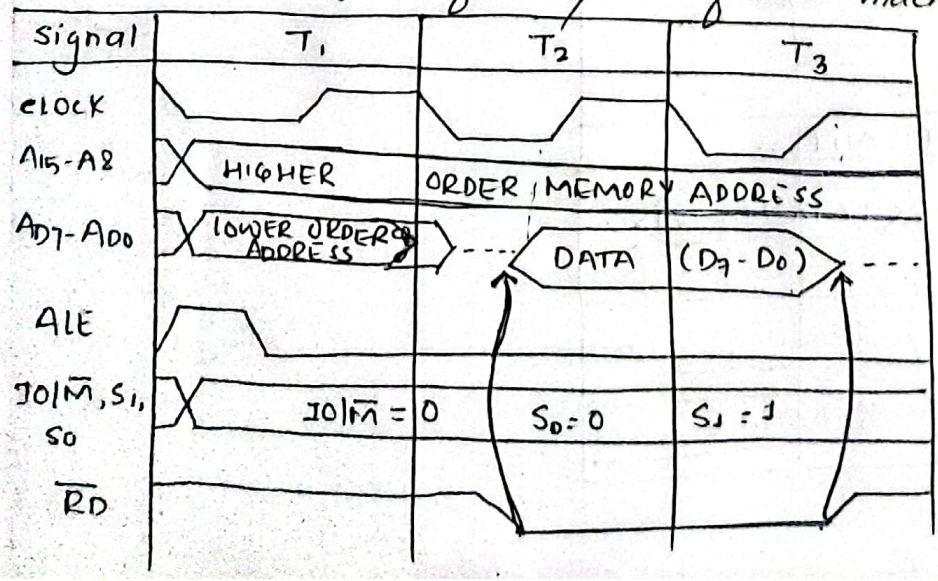


Memory Read Machine cycle of 8085.

The memory read machine cycle is executed by the processor to read a data byte from the memory.

The processor takes 3T states to execute this cycle.

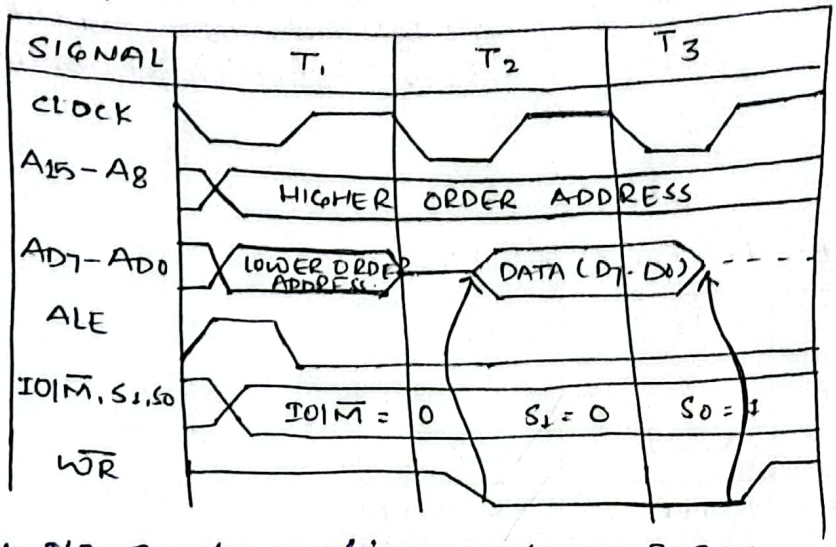
The instruction which has more than one byte word size, use this machine cycle after opcode fetch machine cycle.



3. memory write machine cycle in 8085.

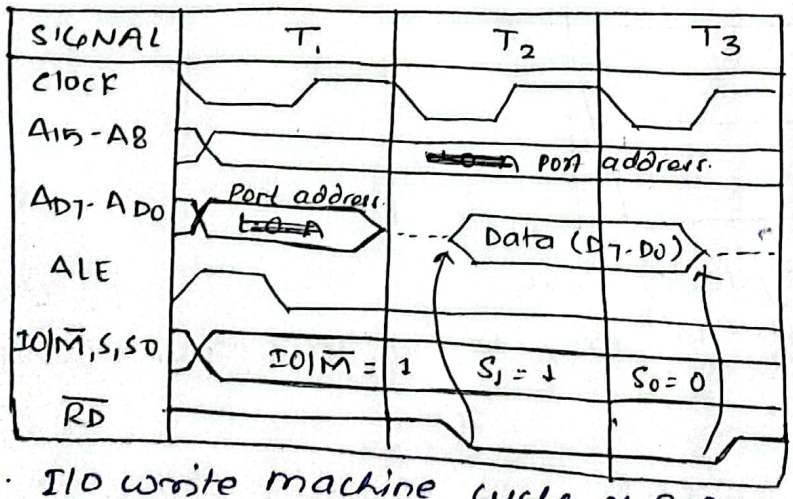
↳ The memory write machine cycle is executed by the processor to write data byte in a memory location.

↳ The processor takes 3T states to execute this machine cycle.

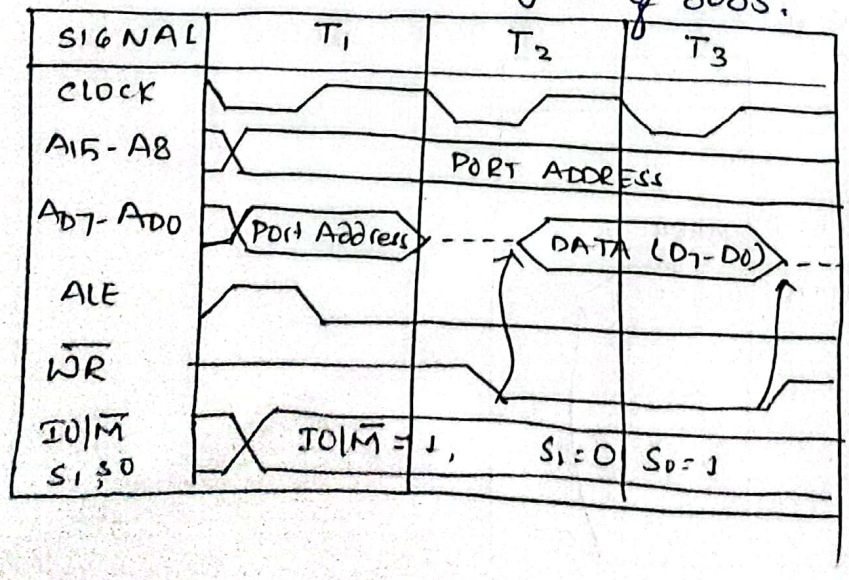


4. I/O Read machine cycle of 8085:

↳ The I/O Read machine cycle is executed by the processor to a data byte from the I/O port or from the peripheral, which mapped in the system.



5. I/O write machine cycle of 8085:



Note:

Timing diagram of MOV R1, R2

only timing diagram of opcode fetch.

Instructions having similar timing diagram:

MOV R1, R2	all ADD R
ADC R	CMA, CMC, STC
CMP R	all INR R
DCR R	all ORA R, ANA R, XRA R
all rotate instructions	all SUB R
SBB R	RIM, SIM
DAA	XCHG.
NOP	

. Timing diagram of MVI A, 32H.

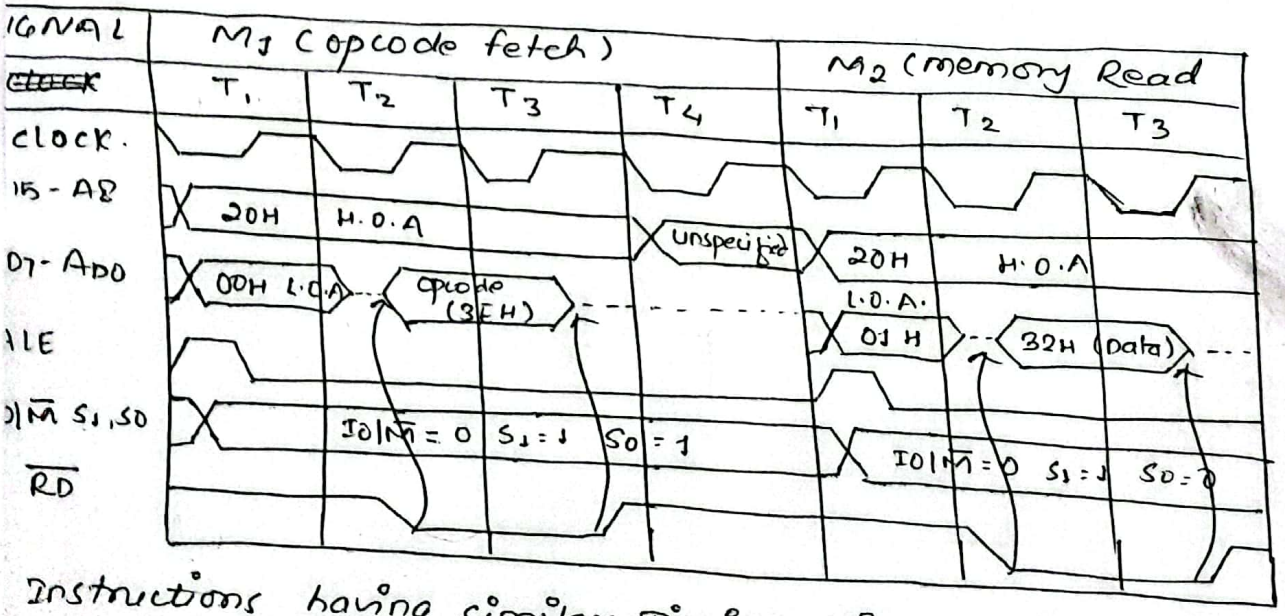
Assume memory address as:

2000 H	3E H (opcode)
2001 H	32H (data)

e know, no. of T-state = 7

opcode fetch = 4T

memory Read = 3T



Instructions having similar Timing Diagram like MVI A, 32H:

ADI Data	ANI Data
ACI Data	ORI Data
UI Data	XRI Data
BI Data	MVI R, Data.

mportant Instructions:

MOV A, C - opcode fetch

MVI A, 33H - OF + MR

LXI B, 2065H - OF + MR + MR

STA C00AH - OF + MR + MR + MW

MOV A, M - OF + MR

MOV M, A - OF + MR

7) IN 84H - OF + MR + I/O Read

8) OUT 80H - OF + MR + I/O write

9) STAX B - OF + MW.

10) LDA 2050H - OF + MR + MR + MR.